

TABLE OF CONTENTS

1. Abstract
2. Revision History
 - 2.1 Changes in Version 1.5
 - 2.2 Changes in Version 2.0
 - 2.3 Changes in Version 2.5
 - 2.4 Changes in Version 3.0
 - 2.5 Changes in Version 3.5
 - 2.6 Changes in Version 4.0
 - 2.7 Changes in Version 5.0
3. Problem Statement
 - 3.1 Business Background
 - 3.2 Needs
 - 3.3 Objectives
4. Requirements
 - 4.1 User Requirement
 - 4.1.1. Glossary of Relevant Domain Terminology
 - 4.1.2. User Groups
 - 4.1.3. Project Context
 - 4.1.4. Functional Requirements
 - 4.1.4.1. Project Scope
 - 4.1.4.2. User Scenarios
 - 4.1.4.3. User Functional Requirements
 - 4.1.5. Non-Functional Requirements
 - 4.1.5.1. Product: Usability Requirements
 - 4.1.5.2. Product: Performance Requirements
 - 4.1.5.3. Product: Availability/Reliability/Security
 - 4.1.5.4. Organizational: Development Requirements
 - 4.1.5.5. Organizational: Operational Requirements
 - 4.1.5.6. Organizational: Environmental Requirements
 - 4.1.5.7. External: Legislative Requirements on Safety/Security
 - 4.1.5.8. External: Cultural and Social Requirements
 - 4.1.5.9. External: Interoperability Requirements
 - 4.2 System Requirements
 - 4.2.1. Functional Requirements
 - 4.2.1.1. System Functional Requirements
 - 4.2.1.2. Data Requirements

4.2.2. Non-Functional Requirements

4.2.2.1. Product: Usability Requirements

4.2.2.2. Product: Performance Requirements

4.2.2.3. Product: Availability/Reliability/Security

4.2.2.4. Organizational: Development Requirements

4.2.2.5. Organizational: Operational Requirements

4.2.2.6. Organizational: Environmental Requirements

4.2.2.7. External: Legislative Requirements on Safety/Security

4.2.2.8. External: Cultural and Social Requirements

4.2.2.9. External: Interoperability Requirements

4.3 Requirements Trace Table

5. Exploratory Studies

5.1. Relevant Development Frameworks

5.2. Relevant Solution Techniques

5.3. Broader Impacts

6. System Design

6.1. Architectural Design

6.2. Structural Design

6.3. User Interface Design

6.4. Behavioral Design

6.5 Design Alternatives & Decision Rationale

7. System Implementation

7.1 Programming Languages & Tools

7.2 Coding Conventions

7.3 Code Version Control

7.4 Implementation Alternatives & Decision Rationale

7.5 Consistency of Code Implementation on Chosen Design Patterns

7.6 Analysis of Key Algorithms

8. System Testing

8.1 Test Automation Framework

8.1.1 Steps for Installing Test Framework

8.1.2 Steps for Running Test Cases

8.2 Test Case Designs

8.3 Test Execution Reports

8.4 Meeting Functional Requirements

8.5 Meeting Non-Functional Requirements

9. Challenges & Open Issues

9.1 Challenges Faced in Requirements Engineering

9.2 Challenges Faces in System Development

9.3 Open Issues & Ideas for Solutions

10. System Manuals

10.1 Instructions for Systems Development

10.1.1 How to set up development environment

10.1.2 Notes on system further extension

10.2 Instructions for System Deployment

10.2.1 Platform Requirements

10.2.2 System Installation

10.3 Instructions for System End Users

11. Conclusion

11.1. Achievement

11.2. Lessons Learned

11.3. Acknowledgement

12. References

Appendix U

Appendix R

Appendix T

Appendix TE

Appendix PM

1. Abstract:

This project's focus is on developing a versatile system that will be capable of navigating indoor spaces. The system's goal is to accurately lead a user to their intended destination by creating paths to destinations in predetermined environments such as restaurants or large department stores. The process of creating this system includes implementing a SLAM model using a Rubik Pi 3 development board, which is mounted to a WaveRover robot. The SLAM model will use information from the Lidar sensor and IMU connected to the Rubik Pi to map the area. The system will also use an iOS app to receive voice commands from the user and use Bluetooth connections to ensure the user is in an appropriate range from the robot. We will also implement text to speech to allow the robot to communicate with the user.

Our goal is for the product to be user-friendly and efficient in its navigation capabilities. The product should provide a more accessible approach for navigating these environments. Therefore, our objectives for the project are to implement accurate mapping, obstacle avoidance, speech recognition, and text to speech. The project will implement all these functions using the Rubik Pi 3 and hardware such as a Wave Rover, Lidar sensor, speaker, and a 9 axis IMU. The project will add to the limited existing projects and information involving the newly released Rubik Pi 3. Additionally, the project may add information regarding the Rubik Pi chipset's compatibility with SLAM models such as the open-source library Slam Toolbox. In addition to adding to existing research, our project may be modified and applied to applicable environments in the future as a helpful and versatile navigational tool for indoor environments. This report will cover the problem statement, user functional/nonfunction requirements, system function/non-functional requirements, exploratory studies, system design, system implementation, system testing, challenges, system manuals, and references. All user and system requirements, test cases, and use cases can be found in the appendices of this report.

2. Changes in Revision History

2.1 Changes in Version 1.5

Updated user functional requirements (Section 4.1.5) and their corresponding system requirements (4.2.2)

Updated Block diagram (Figure 1), Use Case diagram (Figure 2), and Architecture diagram (Figure 3)

Revised Use Case tables UC-001, UC-002, UC-003, UC-004

Added and Updated Trace Table to Section 4.3

2.2 Changes in Version 2.0

Changed Business Background to replace mentions of on-device voice detection to using the iOS application as well as replacing mentions of the rear camera and facial recognition with the Bluetooth signal connected to the iOS app.

Changed Use Case tables UC-001, UC-002, and UC-004 to suit the changes in design with the addition of the mobile application handling what was previously on device voice recognition as well as Bluetooth connection being used instead of a rear camera to detect the user's proximity to the robot (Section 4.1.4.2).

Changed Block Diagram to suit the changes with the addition of a new block for the iOS and iOS device used to execute the voice recognition side of the code as well as shrinking the block for the Rubik Pi to not include functionality it no longer needs due to the switch to using a mobile application to handle some of the tasks.

Changed explanation of requirements: UF-B, UF-F, SF-B-02, SF-D-01, SF-D-02, SF-F-01, SF-F-02, SP-03-02, SP-03-04, SO-01-01, and SE-01-01 to account for the switch to using an iOS app instead of voice recognition and face detection.

Changed Architectural UML diagram to include structural design patterns.

Changed the Trace Table to account for updated requirements.

Changed section 5.2 Relevant solution Techniques to account for new tools and libraries used

Changed section 5.3 Broader Impacts to include the mobile application.

Added sections 6.2, 6.3, 6.4, 6.5, 7.4, section 8 (8.1 - 8.3), 9.2, and 9.3

Updated section 4.1.1. Glossary of Relevant Domain Terminology to include definitions of new tools such as swift

Changed section 10.1.1 How to set up development environment to include directions for using Qualcomm Launcher

Updated References to include new tools such as Swift libraries and removed irrelevant references to unused SDKs

2.3 Changes in Version 2.5

Changed all references of IOS to iOS

Added UML diagrams for structural design patterns to section 6.2

Updated state diagram in section 6.3

2.4 Changes in Version 3.0

Changed description of SF-D-02 in the document to reflect that we are no longer using orb slam and are now using nav2 and slam toolbox.

Changed UC-003 to include returning to the base station since that is a key part of the navigation system

Changed SF-D-01 to remove camera and add lidar for obstacle detection

Added TS-003 for lidar sensor testing

Added TC-006 to test accuracy of the distances the lidar gives for objects it detects

Added TS-004 to test IMUs ability to detect changes in direction

Added TC-007 and TC-008 to test right and left turns change in direction on the IMU respectively.

Updated Section 4.1.3 Figure 1: Block Diagram to reflect software libraries used and LiDAR sensor

Updated Trace Table in Section 4.3 to reflect updated requirements

Updated Section 9.1, 9.2, 9.3

Updated 8.1.1 to add what is necessary for the test framework with the addition of new test cases

Updated 8.1.2 to add new steps to test hardware with the addition of new test cases

Updated Sections 8.2 and 8.3 to add descriptions of the new test cases

Updated sections 6.1 and 6.2 to add new Model-View- Viewmodel Pattern and updated the classes

2.5 Changes in Version 3.5

Updated test execution reports. Added failed test cases for each test case for the time where the batteries were dead in the robot, for the slidar node not working correctly, and the app not correctly picking up a keyword as well as the fixes.

Added missing system and user requirements for each test case.

Fixed architecture diagram and added to description

2.6 Changes in Version 4.0

Updated Section 8.2 and 8.3 to add TS-005, TS-006 and TC-009 and TC-010 adding tests for obstacle avoidance and path following.

Added sections 7.5 Consistency of Code Implementation on Chosen Design Patterns and 7.6 Analysis of Key Algorithms

Updated sections 9.2 Challenges Faced in System Development and 9.3 Open Issues and Ideas for Solutions

Added Sections 10.1.2 Notes on system further extension, 10.2 Instructions for System Deployment, 10.2.1 Platform Requirements, 10.2.2 System Installation and 10.3 Instructions for System End Users

Added Appendix PM Project Management Report

2.7 Changes in Version 5.0

Added Section 11 Conclusion, 11.1 Achievement, 11.2 Lessons Learned, 11.3 Acknowledgement

Revised Test execution reports

3. Problem Statement

3.1. Business Background

Currently there are not many tools that can be used for indoor navigation, and such tools that are available are not commonly used. Navigating large indoor spaces can be time-consuming as well as tracking down another person (e.g. Store attendant) to locate the desired location or item you are searching for. Our project will bridge this gap in available navigation tools by implementing a voice activated robot that will be able to navigate a predetermined environment like a department store or school building. The robot will have the map of its environment predetermined but will be actively scanning for new obstacles that might be in its way. The user will be able to use an iOS app to connect to the robot. The user will voice a keyword through the app, once recognized the user will be able to voice commands to the robot via the app. The robot will also use text to speech to communicate with the user. The robot will use a Lidar sensor to perform obstacle detection as well as generate a map in real time. This allows for optimal processing time while the robot is completing a task. The robot will also pause navigation when the user is out of range, using the iOS Bluetooth connection to the robot. Once the user arrives at the destination, the robot will signal to the user that the destination has been reached and return to idle mode to end the task. When in idle mode, the robot will simply start moving towards a predetermined spot considered home base, which is also where customers will be able to find our robot. This diminishes the search time for an actual employee. Our robot will improve efficiency and become a helpful tool in a variety of establishments.

3.2. Needs

Our project will use a voice activated robot to make navigation in large indoor spaces easier and more accessible for the user. Searching for an item when shopping can be frustrating and this product aims to reduce that frustration a customer can have while shopping. Features like voice recognition and text to speech will allow the user to request directions from the robot using an iOS app and this will bypass the need to track down a store attendant or use some other navigational tool to find the user's desired location. The SLAM model will create a map of the robot's surroundings and include obstacles or obstructions. Our navigation software using this map will avoid any obstacles and lead the user to the destination using the most accessible path. Our robot will have pre-mapped destinations to lead the user to. The user will access the robot using an iOS app to voice a key word, afterwards the user can request directions to the pre-determined destinations. If the user exits the range of the robot, the robot will pause all functions and wait for the user to be back in range before continuing.

3.3 Objectives

The objectives of this project include implementing accurate voice recognition, accurately generating maps, accurately avoiding obstacles, and leading the user to the correct location. By implementing the intended features, this project should cut down the time the user spends locating an item/location. By implementing accurate voice recognition, it will reduce the risk of the user being frustrated with the robot. By implementing accurately generated maps, the robot will be able to follow the most efficient route possible to guide the user in the least amount of time. By accurately avoiding obstacles, there will not be a risk of the robot getting stuck on an object or running into a shelf that was not on the pre-generated map. By leading the user to the correct location, the user will save lots of time finding the item they were searching for while shopping.

4. Requirements

4.1. User Requirement

4.1.1. Glossary of Relevant Domain Terminology

iOS : The iPhone operating system created by apple for apple devices.

Rubik Pi 3: A powerful and compact development board designed for edge AI and robotics, built on the Qualcomm QCS6490 processor

ORB-SLAM 3 (Simultaneous Localization and Mapping): A computational process that allows an autonomous device to build a map of an unknown environment while simultaneously keeping track of its own location within that map.

Wave Rover Robot: A flexible and expandable four-wheel drive mobile robot chassis. Designed for secondary development and capable of handling multiple and complex terrains.

Swift: An open-source programming language used to develop applications on iOS devices

SFSpeechRecognizer: A Swift library designed for speech recognition on an iOS application

Bluetooth Playground: A Swift library to work with iOS Bluetooth connections

Voice Recognition: Software that allows the system to understand and parse through spoken language

Text to Speech (TTS): Software that reads from written text and relays the message vocally through a speaker

Robot Operating System (ROS): Software libraries and tools that can be used to build a robot.

Slam Toolbox: An open source Ros2 library for 2D simultaneous localization and mapping(SLAM)

Navigation2 (Nav2): An open source Ros2 library for navigation software that allows a robot to get from point A to B

LiDAR (Light Detection and Ranging) sensor: for this project we are using a 2D RPLiDAR sensor. The sensor uses invisible lasers from 360 degrees and measures distances by calculating the time it takes for the lasers to reflect off objects and return to the sensor.

IMU (Inertial Measurement Unit): A sensor that measures acceleration and angular rate. It uses a combination of accelerometers, gyroscopes, and magnetometers.

Teleop Twist Keyboard: A node which converts keys pressed into Twist commands which can be used to control the robot manually.

4.1.2. User Groups

Customers in Large/Warehouse Stores: Needs to locate items or products in the store

Workers in Large/Warehouse Stores: Needs to locate items or products in the store or in the storage in the back

Restaurant Diners: Needs to locate an open seat or a seat that has been reserved

4.1.3. Project Context

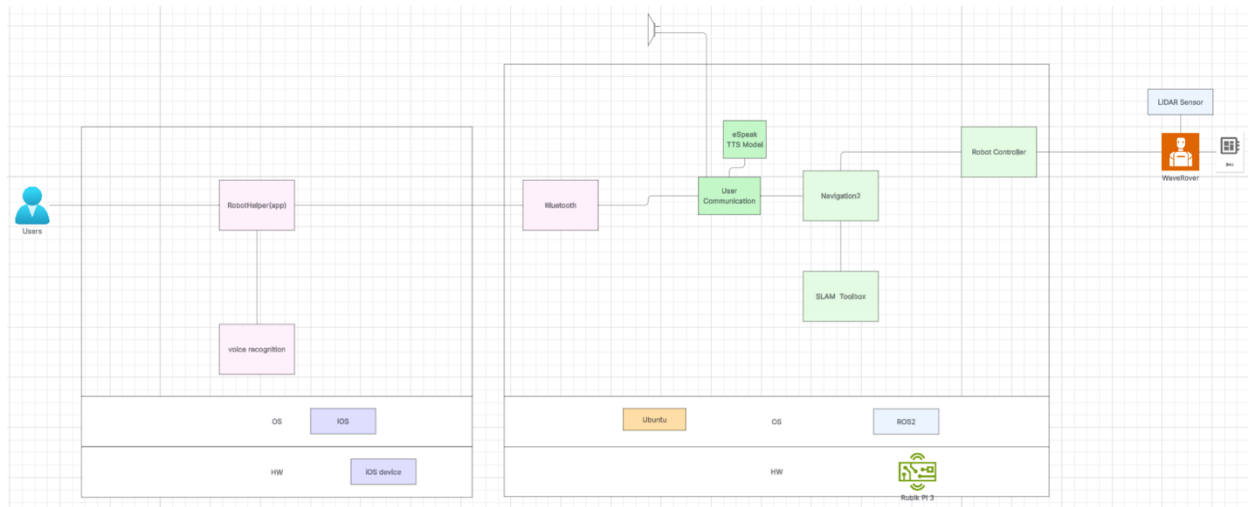


Figure 1: Block Diagram

This context diagram above shows how our hardware will work with our chosen operating systems and models. We will be using the following hardware: an iOS device, a Rubik Pi 3, a SLAMTEC A1 RPLIDAR sensor, Wave Rover robot, a 9 axis IMU with a gyroscope and accelerometer, and a speaker. We will be using an iOS device to host a mobile application that will use voice recognition libraries from Swift. This mobile application will communicate to our Rubik Pi 3 via a Bluetooth connection. The Rubik Pi will use the operating systems Ubuntu and ROS2. The SLAM model will also run on the Rubik Pi and we will use ROS2 for libraries to control our WaveRover robot. Additionally, we will use ROS2 libraries to help navigate our generated routes. A speaker connected to the Rubik Pi will provide feedback to the user via a text-to-speech model.

4.1.4. Functional Requirements

4.1.4.1. Project Scope

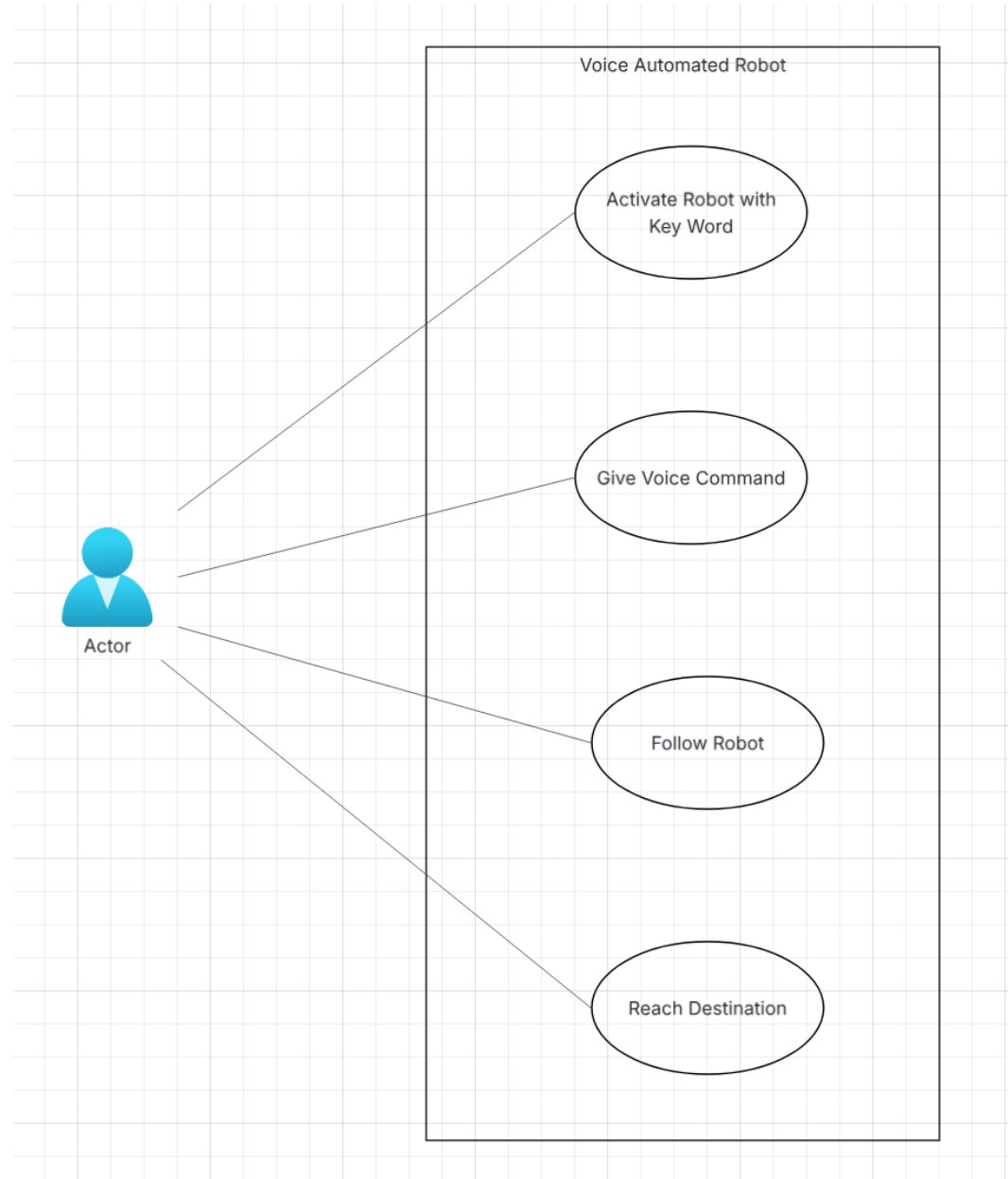


Figure 2: Use Case Diagram

4.1.4.2. User Scenarios

UC-001:

This use case describes when the user wants to interact with the robot. They start by saying a specific activation keyword to the mobile application to become registered as the active user; the mobile application will begin listening for a keyword once the user presses the speak button on the UI. Once it receives this key word, the mobile application communicates with the robot through Bluetooth activated and the user is registered as the current active user through their Bluetooth connection from their phone to the robot. If no valid keyword is detected by the application, the robot will remain idle.

(Refer to Figure 4.2 in Appendix U)

UC-002:

This use case describes how a user provides a destination through voice input. The user will state where they would like to go and if the mobile application successfully recognizes the command, the robot will confirm the destination using TTS and starts moving. If the command is not recognized, the application will print a message to the user stating that no valid keyword was detected, and the application continues to listen for a valid keyword.

(Refer to Figure 4.3 in Appendix U)

UC-003:

This use case describes how the robot directs the user to their intended destination. Using a pre-generated map and SLAM models, the robot will follow the most optimal route while adapting to any environmental changes or obstacles. If an obstacle is detected, the robot will reroute or return to the starting point as needed. The process will end when the robot successfully reaches the starting station.

(Refer to Figure 4.4 in Appendix U)

UC-004:

This use case describes how the robot will voice directions to the user to ready them before it turns left or right. This allows the user to be ready for abrupt movements and reduces the risk of the user falling behind and exiting the Bluetooth range. It is linked to functional requirement UF-A and system requirements SF-A-01 and SF-A-02.

(Refer to Figure 4.5 in Appendix U)

4.1.4.3. User Functional Requirements

UF-A: The user will be able to follow the robot to get to their desired destination. The robot will travel at a comfortable speed and allow the user a reasonable amount of time to respond to any change in direction.

(Refer to Figure 4.6 in Appendix R)

UF-B: The user will be able to issue voice commands to the mobile application. When they match registered keywords, the system will engage the robot's navigational tool. The mobile application ensures commands are understood in real time.

(Refer to Figure 4.7 in Appendix R)

UF-D: The user will be able to safely follow the robot as it travels through the environment, with the robot detecting and avoiding any obstacles in the path. This will allow the user to travel to their destination efficiently and without collisions.

(Refer to Figure 4.8 in Appendix R)

UF-F: The user will be able to activate the robot by using a keyword recognized by the mobile application. This creates a simple interaction needed to activate the robot, allowing the user to begin navigation through manual input.

(Refer to Figure 4.9 in Appendix R)

4.1.5. Non-functional Requirements

4.1.5.1. Product: Usability Requirements

UP-03: The robot navigation tool will be intuitive and easy to use, allowing users to interact with simple voice commands and clear feedback. [OBJ]

(Refer to Figure 4.10 in Appendix R)

4.1.5.2. Product: Performance Requirements

4.1.5.3. Product: Availability/Reliability/Security

4.1.5.6. Organizational: Operational Requirements

4.1.5.8. External: Cultural and Social Requirements

UE-01: The robot will avoid becoming an obstacle in the environment it is designed to help navigate. Therefore, reducing any potential harm it may cause any people nearby.

(Refer to Figure 4.11 in Appendix R)

4.2. System Requirements

4.2.1. Functional Requirements

4.2.1.1. System Functional Requirements

SF-A-01: The system will generate and execute an optimal path from the robot's location to the specified destination using mapping and navigation algorithms. This allows the robot to travel the most efficient route to the user's destination. 

(Refer to Figure 4.12 in Appendix R)

SF-A-02: The system will be able to generate consistent messages throughout the navigating process to provide directions and confirmations to the user through text to speech. This allows the robot to keep the user updated as it focuses on moving towards the destination.

(Refer to Figure 4.13 in Appendix R)

SF-B-01: The system will use voice recognition capable of detecting and interpreting spoken commands from the user. This enables the system to communicate with the user and initiate actions from verbal commands.

(Refer to Figure 4.14 in Appendix R)

SF-B-02: The application will take input speech from the user and generate text from the voice input. If a specific keyword is detected by the application, it will activate the robot through Bluetooth and begin navigation to the users' desired location. This allows the system to react to the user through the application detecting voice.

(Refer to Figure 4.15 in Appendix R)

SF-B-03: The system will be able to only detect and take key words from one user. This ensures that the system will only focus on one user when navigating through crowded environments.

(Refer to Figure 4.16 in Appendix R)

SF-D-01: The system will use the ORB SLAM model using sensors such as cameras to identify obstacles in the environment. This enables the system to respond to hazards and ensure safe and interrupted navigation for the user.

(Refer to Figure 4.17 in Appendix R)

SF-D-02: The system will be able to utilize Slam Toolbox and NAV2 models to adjust the robot's path as it encounters an obstacle. This allows the robot to not get stuck anywhere on its path to its destination while also not delaying time by stopping.

(Refer to Figure 4.18 in Appendix R)

SF-F-01: Once the system receives the instructions from the mobile application, it will automatically start running the programs required to complete a navigational task. This ensures the robot will be easily accessible anytime a user needs help.

(Refer to Figure 4.19 in Appendix R)

SF-F-02: The application will register the user as the current active user after connecting to their mobile device through Bluetooth and recognizing the activation keyword. It will then transition to listening for a desired location keyword. This ensures the robot will respond promptly and accurately to user commands.

(Refer to Figure 4.20 in Appendix R)

4.2.1.2. Data Requirements

The data required to achieve these functional requirements includes data specific to the environment. The environment will be pre-mapped and will use a SLAM model such as SLAM Toolbox to identify obstacles in real time that were not on the pre-generated map. The mobile application will require audio input from the user in order to activate the system. When a predefined keyword is detected by the application, the user will be recognized as the current user and navigation will begin with the user being detected by linking their mobile device running the application through Bluetooth. The user will also need to provide an action such as a request to a location so that the system can create a path.

4.2.2. Non-functional Requirements

4.2.2.1. Product: Usability Requirements

SP-03-01: The system will recognize and interpret key user commands. This allows the robot to appropriately respond to user commands with the correct actions.

(Refer to Figure 4.21 in Appendix R)

SP-03-03: The system will be able to generate and output an appropriate response to the user within three seconds of the command from the user being inputted. This allows for efficiency between the command and the completion of the actual task, not letting the user wait around.

(Refer to Figure 4.23 in Appendix R)

SP-03-02: The system will respond to the event of a user exiting the Bluetooth range in a reasonable short amount of time. If the Bluetooth signal is dropped, then the robot will stop all functions within one second. This ensures that the robot will not venture too far away from the last known location of the user once they are out of signal range.

(Refer to Figure 4.22 in Appendix R)

SP-03-04: The system will be able to successfully use Bluetooth connection to identify the user

within five seconds of the user connecting to the robot through the mobile application. This ensures that the user can easily activate the robot by simply connecting through a Bluetooth device with the application installed.

(Refer to Figure 4.24 in Appendix R)

4.2.2.5. Organizational: Operational Requirements

SO-01-01: The system will support one active user at a time with Bluetooth connectivity through the mobile application, ensuring there is only one user giving commands to the system at a time. This will prevent command conflicts and maintain a personalized experience for the user.

(Refer to Figure 4.25 in Appendix R)

4.2.2.8. External: Cultural and Social Requirements

SE-01-01: The robot, after losing connection to the user, will go back to idle mode and wait for either a new user or that same user to reconnect to the robot through Bluetooth and give a new destination. After a certain amount of time, the robot will return to its starting station. This will make sure the robot will not trip any customers walking through the area the robot was left in. This will also prevent the issue of the robot being stuck waiting for a user to reconnect in the event they walk away and find something else.

(Refer to Figure 4.26 in Appendix R)

4.3 Requirements Trace Table

Project Name: Voice Activated Guide Robot			
User Requirements		System Requirements	
Req ID	Description	Req ID	Description
UE-01	The robot will not obstruct other users paths when not in use.	SE-01-01	The robot will return to the starting station after completing navigation or after user is out of view for over a minute without returning.
UF-A	The user can follow a robot navigational tool to the desired location.	SF-A-01	The system will create and execute a path from the robots current location to the destination.
		SF-A-02	The system shall use text to speech to provide directions and feedback for the user to follow during navigation.
UF-B	The user will be able to issue voice commands to the mobile application, which when they match registered keywords will engage the robot's navigational tool.	SF-B-01	The system will use a voice recognition module to detect and interpret spoken commands.
		SF-B-02	The web application will parse through spoken commands for navigation actions.
		SF-B-03	The system shall recognize only one bluetooth connected user.
UF-D	The user can follow the robot without running into obstacles such as objects in the middle of the path.	SF-D-01	The system will use a SLAM model and sensors such as lidar to identify obstacles in the path.
		SF-D-02	The system shall automatically adjust the path to avoid obstacles.
UF-F	The user speak keywords into the mobile application and be recognized by the system.	SF-F-01	The mobile application shall listen for the predefined keyword after opened
		SF-F-02	The mobile application after recognizing the key word will transition into actively listening for directions.
UP-03	The robot navigation tool shall be easy to use.	SP-03-01	The system shall recognize user commands by communicating with the mobile application to recognize key actions.
		SP-03-02	The system will respond to the user dropping Bluetooth signal by pausing movement within 1 second
		SP-03-03	The system will respond within three seconds of the user issuing a command
		SP-03-04	The system will use Bluetooth connection to identify the user within 1 seconds of connecting
Acknowledgment: Generated from the CapStone process management system ©2026			

5. Exploratory Studies

5.1 Relevant Development Frameworks

Our system consists of both software and hardware components to achieve our goal. Different frameworks will all come together to be the base of our robot that has multiple functions like object detection and mapping as well as a user-friendly interaction feature. A large majority of the system will be operating on the Rubik Pi 3 development board, a newer development board that provides the needed processing power we need for a real-time operating robot [1]. Attached to it will be the Wave Rover Robot, which supports sensor integration as well as motor control and will support the LiDAR sensor and IMU that are required for mapping the area [10]. Our operating system choice will be Ubuntu Linux, a popularly known operating system in the robotics community. We chose this due to its compatibility with ROS2 which provides important open-source packages for our project. The project will be programmed in Python, which we chose because of its compatibility with ROS2 features with easy real-time performance usage. Python is good for coding our software for motion control and implementing our SLAM model, which is the most optimal way to save processing time for our robot. We will use ROS2 to use software libraries to work with the Waverover [11]. ROS provides hardware abstractions that make working with the hardware components more direct. ROS also provides software drivers and algorithms that can be used in our project. We plan on using threading to allow multiple systems to be running at the same time. Overall, this would give us a solid foundation of a flexible development environment that would support everything the robot has to offer.

5.2 Relevant Solution Techniques

One of the first choices on different computational methods to use for this project was whether to focus on a predetermined map or use a SLAM model. For this application, we decided to use the SLAM model SLAM Toolbox from ROS to create a map. This model will provide the solution for how we will reduce processing burdens while still powering multiple modules at once. It will help construct the map required to operate within the space while also keeping track of its own position within that space. SLAM Toolbox is an open-source implementation designed to work directly with ROS and Nav2 and it is compatible with our robot [9]. The Lidar sensor will be able to continuously update the cost map the robot will be using, so it has a better understanding of its surroundings as it moves around the environment. We plan to use an iOS mobile application to handle voice recognition. Swift [7] the open-source programming language developed by apple for iOS devices includes speech recognition libraries such as SFSpeechRecognizer which captures audio and converts that audio to text [2]. This library will allow our mobile application to perform real-time speech detection and listen for user input. This speech recognition library will also provide the necessary tools to parse through the transcribed audio input for key words and destinations. Once the keyword is verified and the destination is confirmed, we will send that information to our development board to begin navigation using our SLAM model. Since we are handling voice recognition and processing on the mobile

application, it will help to limit the models that must run on the Rubik Pi 3 board. Our LiDAR sensor attached to our Rubik Pi will provide data for the SLAM model to determine its position in the environment. The LiDAR sensor will provide data to Nav2 that will allow Nav2 plugins to determine when unexpected obstacles appear in the environment. The LiDAR sensor will remain on and will continue to send information to our system for as long as navigation is in process. Additionally, instead of using face detection models to ensure the user is near the robot, we plan on using a Bluetooth connection from the mobile app to the Rubik Pi. The Rubik Pi documentation specifies how to search for Bluetooth connections and how to pair devices [4]. Swift also provides a library called Playground Bluetooth which provides a framework to search for and connect to devices [8]. We are also planning on using the available libraries in ROS to control the WaveRover[11].

5.3 Broader Impacts

Our project will contribute to tools available for navigation and will offer a navigation tool for smaller indoor spaces. Our project will make navigating an environment like a large grocery store much more accessible and time efficient to the average customer. Our project will be demonstrated in a school building but can be modified and applied to applicable environments or scenarios. Additionally, our project will add to the information available about the Rubik Pi 3 which was recently released for purchase in January 2025. Our project will provide more information on how to use a Rubik Pi's Bluetooth connection in a real-world application. Additionally, our project will provide more information and metrics on the SLAM Toolbox model's efficiency on a Rubik Pi 3 board. Our project will also provide examples of object detection and navigation using available models and open-source libraries, therefore contributing to the existing use cases of these software tools. Our project may be used as a learning tool for future projects and research about the Rubik Pi 3 in addition to the hardware and software tools used.

6. System Design

6.1 Architectural Design

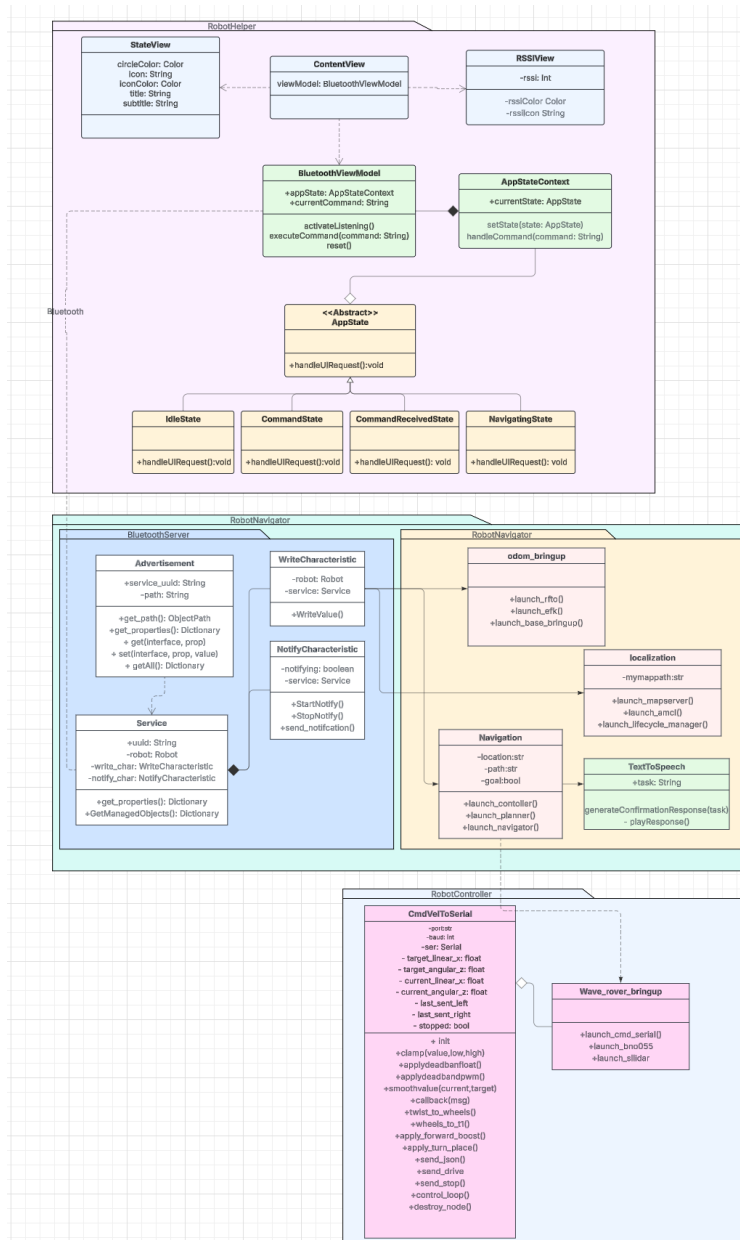


Figure 3: UML Diagram

The architectural design for this system uses layered architecture. The top layer represents the tasks handled by our mobile application, such as speech recognition and using Bluetooth signals to identify the user. This is the layer that the user will interact with when using the mobile application. The mobile application will have four different states such as listening for keywords, listening for command, notifying the user the command was received, and navigation state. This layer will send their intended destination to the layer below which is responsible for mapping and navigation. This layer sends the intended destination to the Rubik Pi using Bluetooth Low Energy (BLE), where the app first discovers the device through advertisement, connects to its GATT service, and writes the command to a write characteristic. The mapping and navigation layer will work with a SLAM model which receives data from the sensors, loads a map of the environment, and creates a route to the destination. This layer will also avoid obstacles in the path using Nav2 the map created by the SLAM model which will continue analyzing the environment when the robot is moving. Using the Bluetooth signal gained from the first layer (mobile app), the mapping and navigation classes will pause navigation when the Bluetooth strength falls below a certain threshold over a certain amount of time. The navigation layer will also provide updates to the route to the user using text to speech. Lastly, this layer will communicate with the last layer Robot via an interface for robot controls which uses an adapter to communicate with the last layer. The last layer is responsible for controlling the robot's movements using the ROS library and will follow commands based on the route gathered from the navigation layer.

6.2 Structural Design

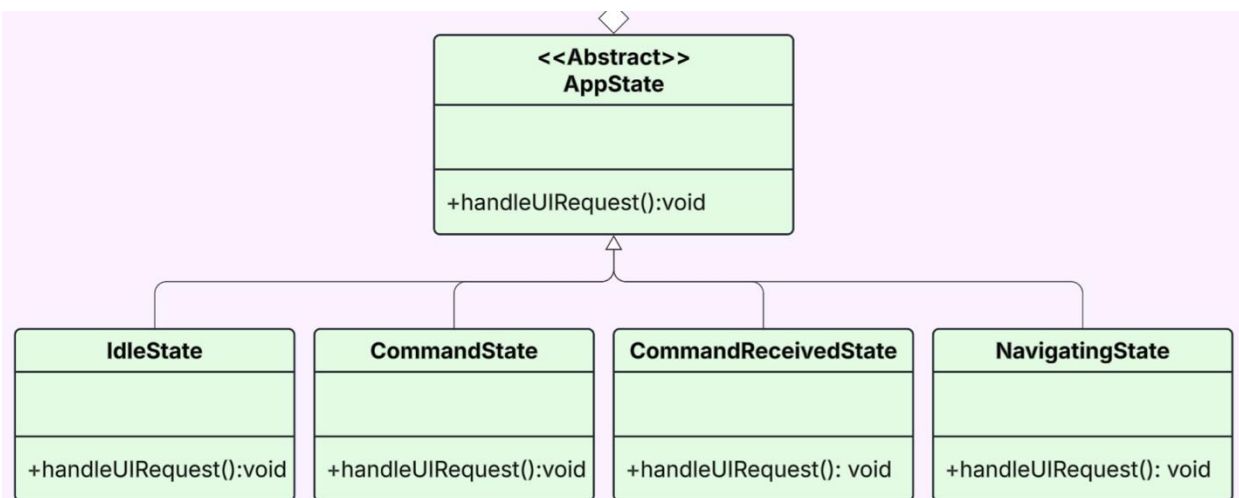


Figure 4: State Design Pattern

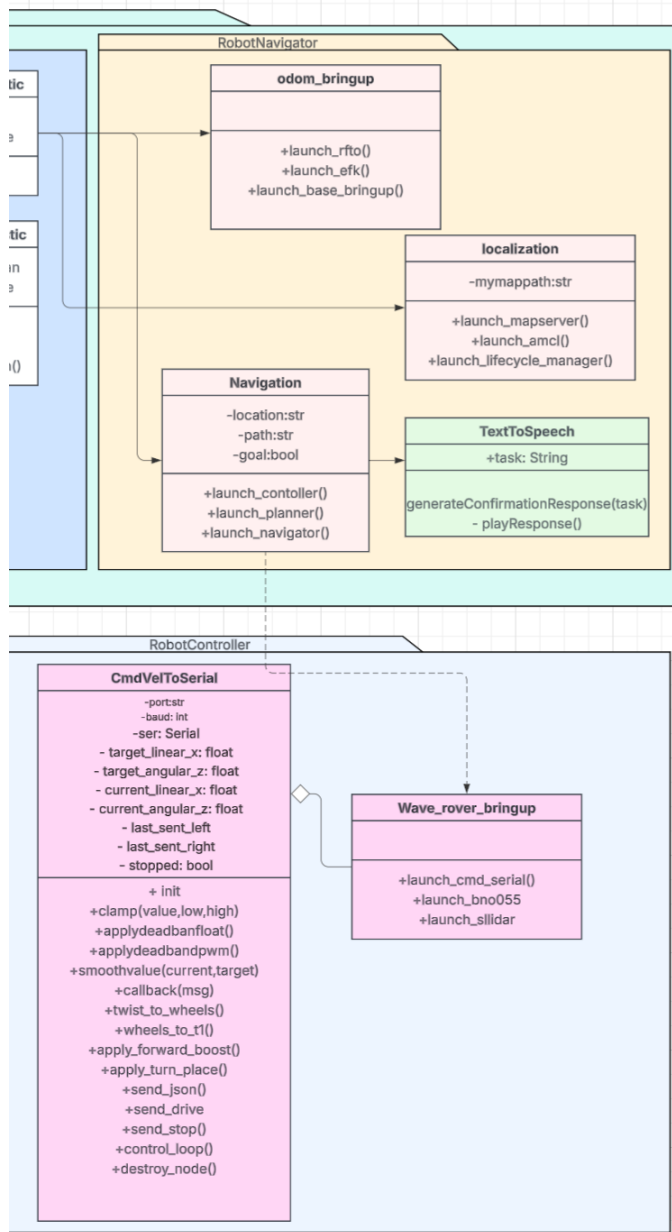


Figure 5: Adapter Design Pattern

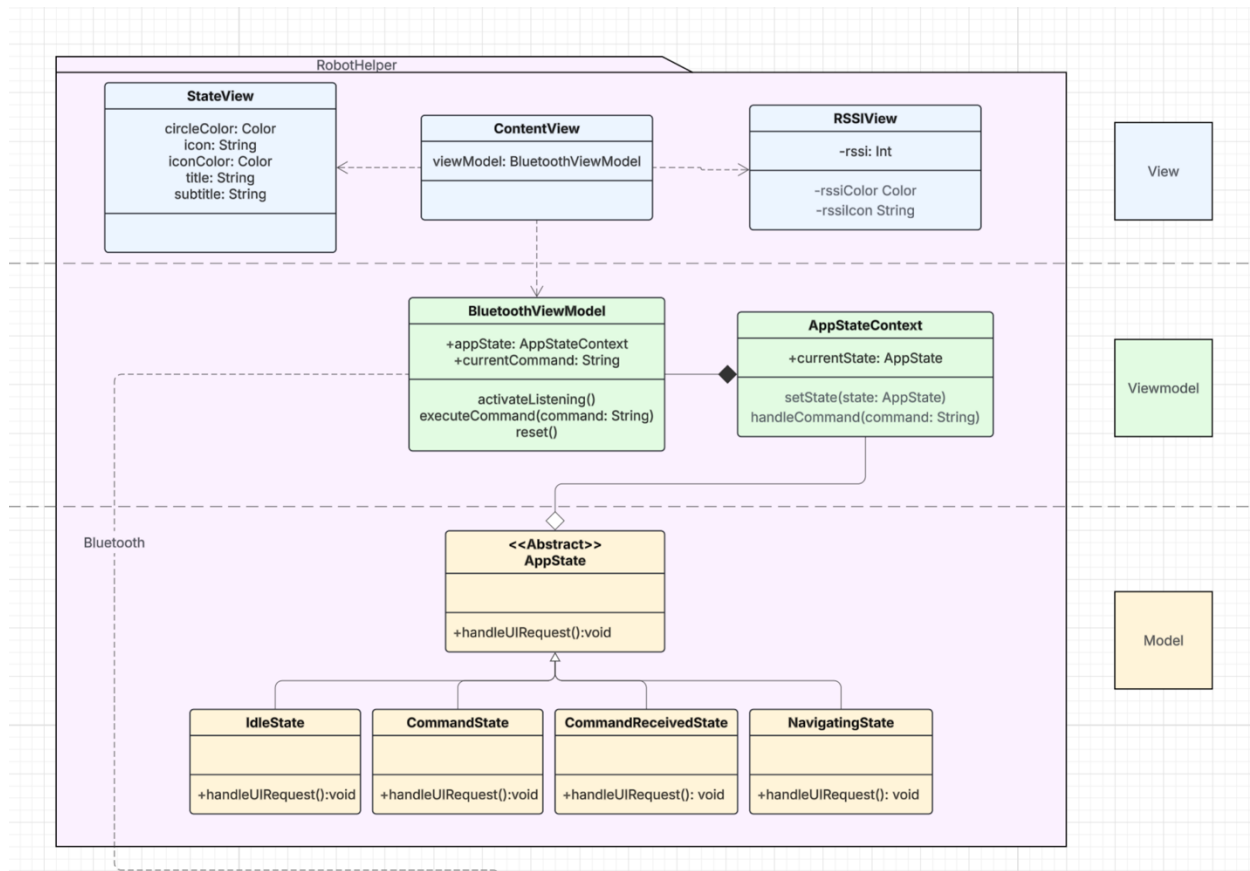


Figure 6: Model-View-ViewModel Design Pattern

We chose to go with the state pattern for one of the structural design patterns. This is due to the way the robot operates. It goes through three different states, for example when in Idle state it waits for a keyword to be spoken before listening for commands. In the Command state, the mobile application actively listens for a valid command that will provide a destination to the robot. The final state is the Command Received state. After a valid command has been given, the mobile application is ready to transfer the instructions to the robot. These states are used by the mobile application to allow the app to process the different states it moves through before passing information on to the Rubik Pi which will control the robot. We decided to include this pattern to easily modify and separate the behavior of the mobile application based on its state. Another pattern we felt was applicable to the project is the adapter structural pattern. This pattern provides a method of using an interface and adapter to communicate with a third-party library or incompatible system. This will allow us to easily write code that can allow us to control the WaveRover through ROS. Lastly, we chose to use the MVVM design pattern for our application because it cleanly separates the UI from the app logic and data, making the app easier to test and maintain. The ViewModel handles Bluetooth connections, commands, and app state, while the View simply reacts to changes, and the Model holds the data to integrate the state design pattern without congesting the UI. For the adapter design pattern, we adapt the JSON commands used by the robot to serial commands which can be used so that Nav2 can control the robot. We decided

to use the adapter pattern since it allows us to adapt our robot movement commands into something which can be used by Nav2 within ROS, allowing us to use that built-in navigation module. It may look through the diagram like these classes connect more closely than they are but this is due to the way ROS topics communicate with each other with publishers and subscribers which can easily be switched out. This adapter of JSON commands to serial was required due to the WaveRover being suited for the Raspberry Pi instead of the Rubik Pi leading to compatibility issues when connecting the pi into the serial connector on the WaveRover.

6.3 User Interface Design

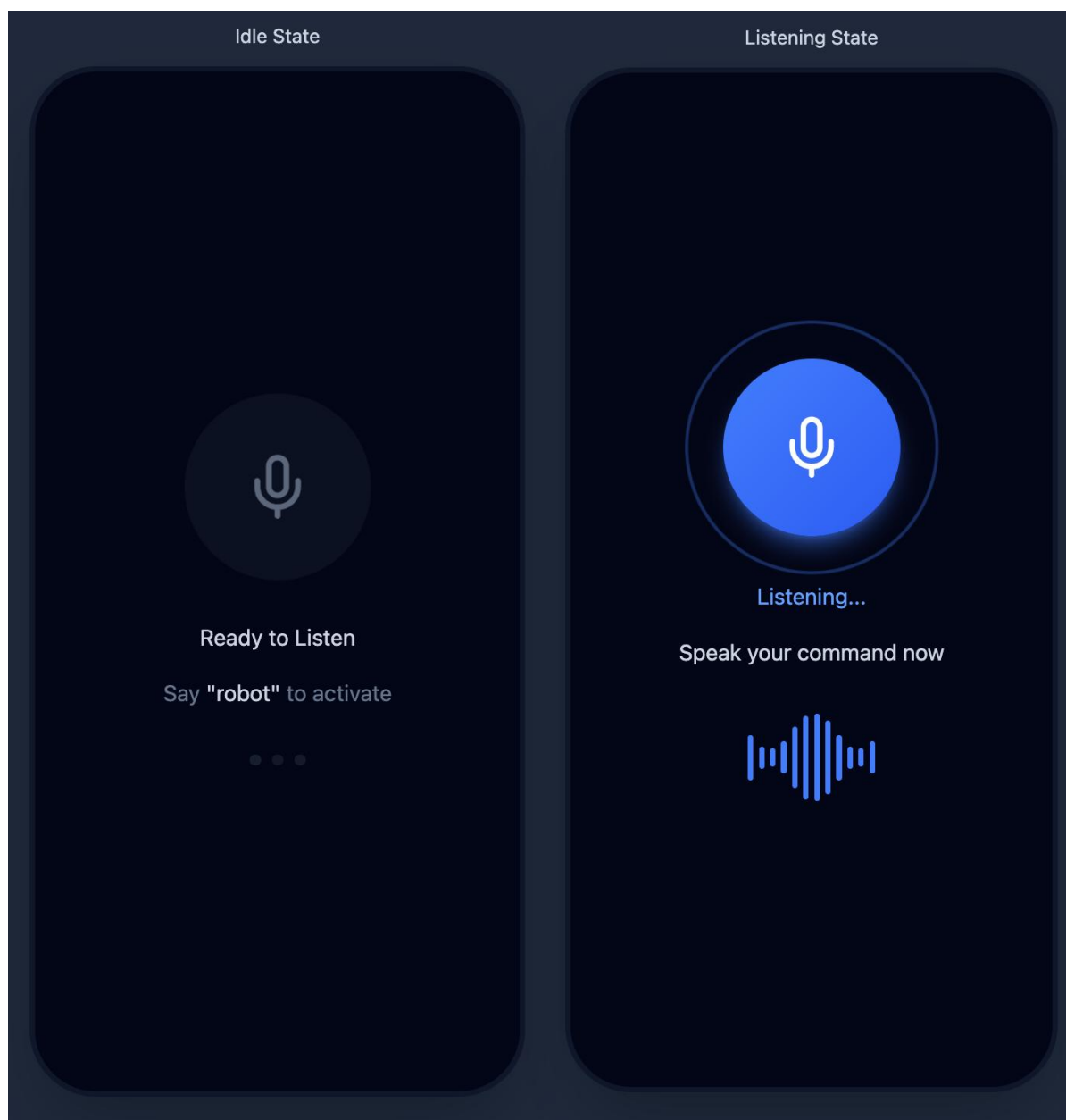


Figure 7: Idle and Listening state

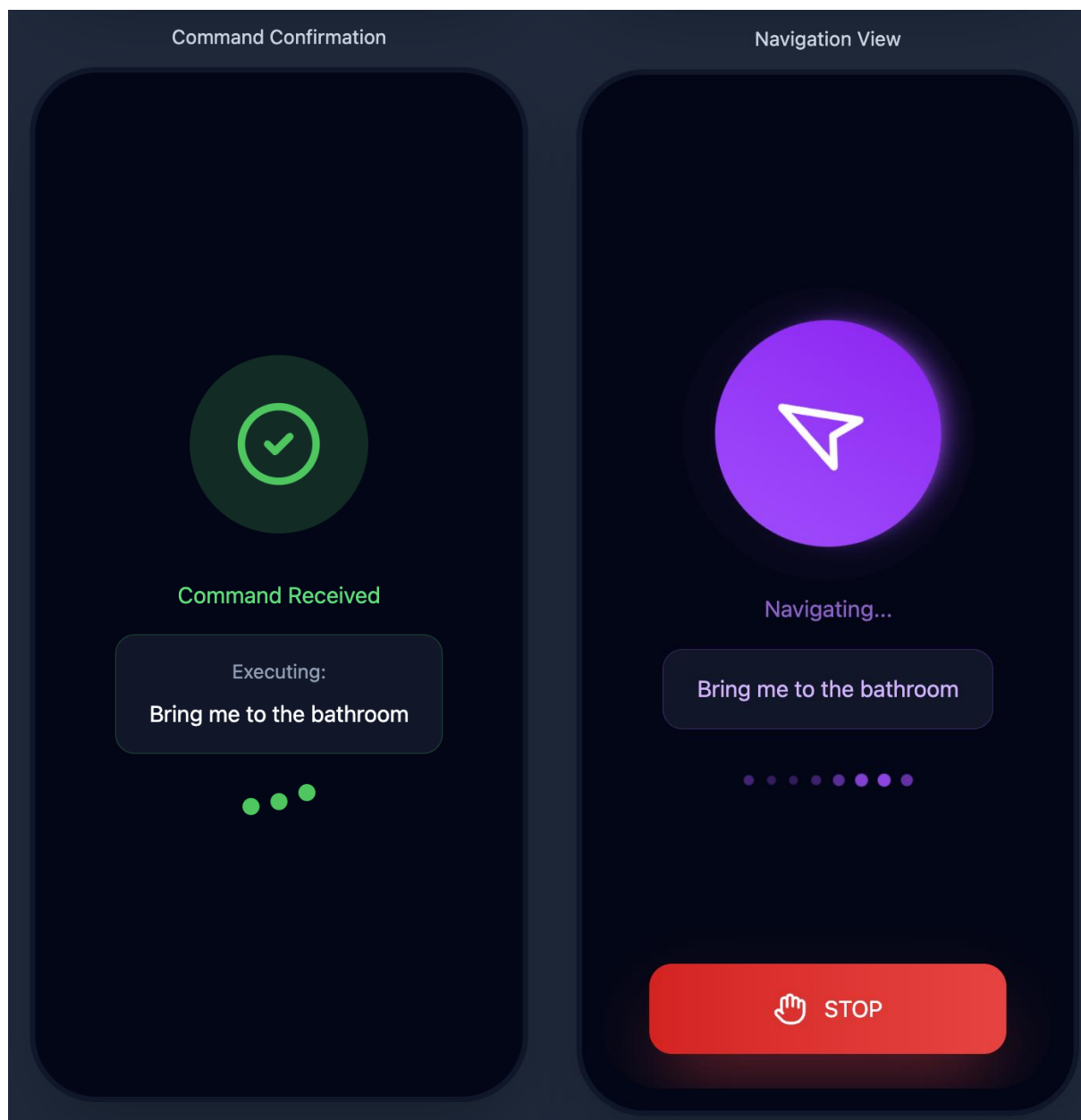


Figure 8: Command Received and Navigation

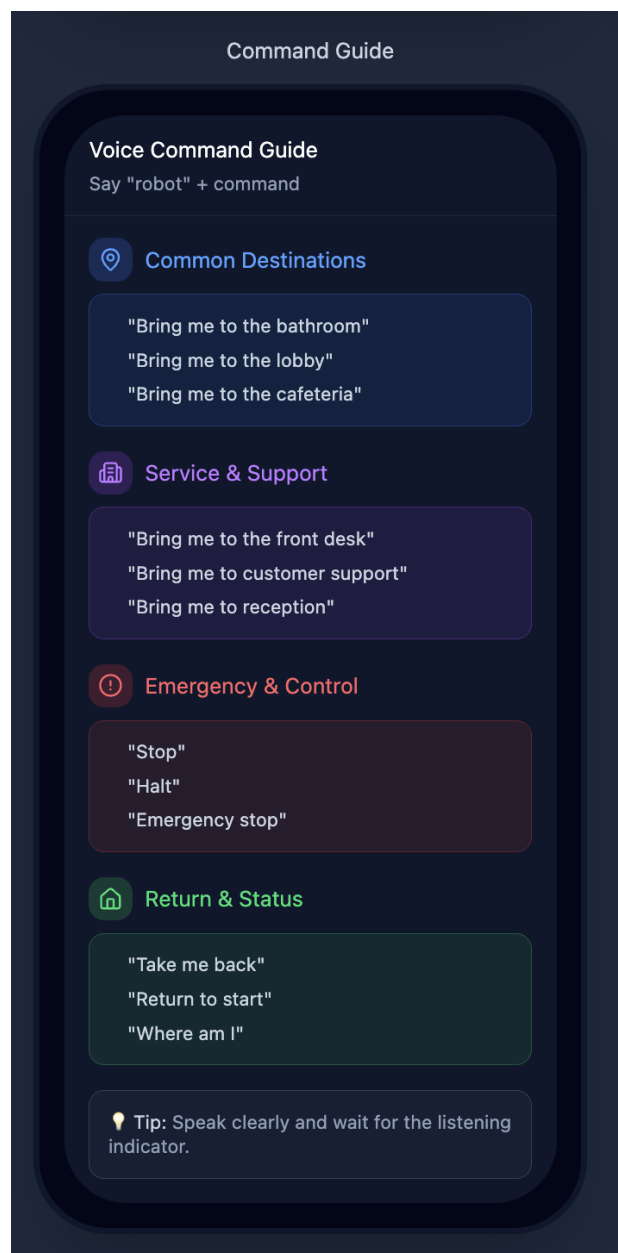


Figure 9: Command Guide

6.4 Behavioral Design

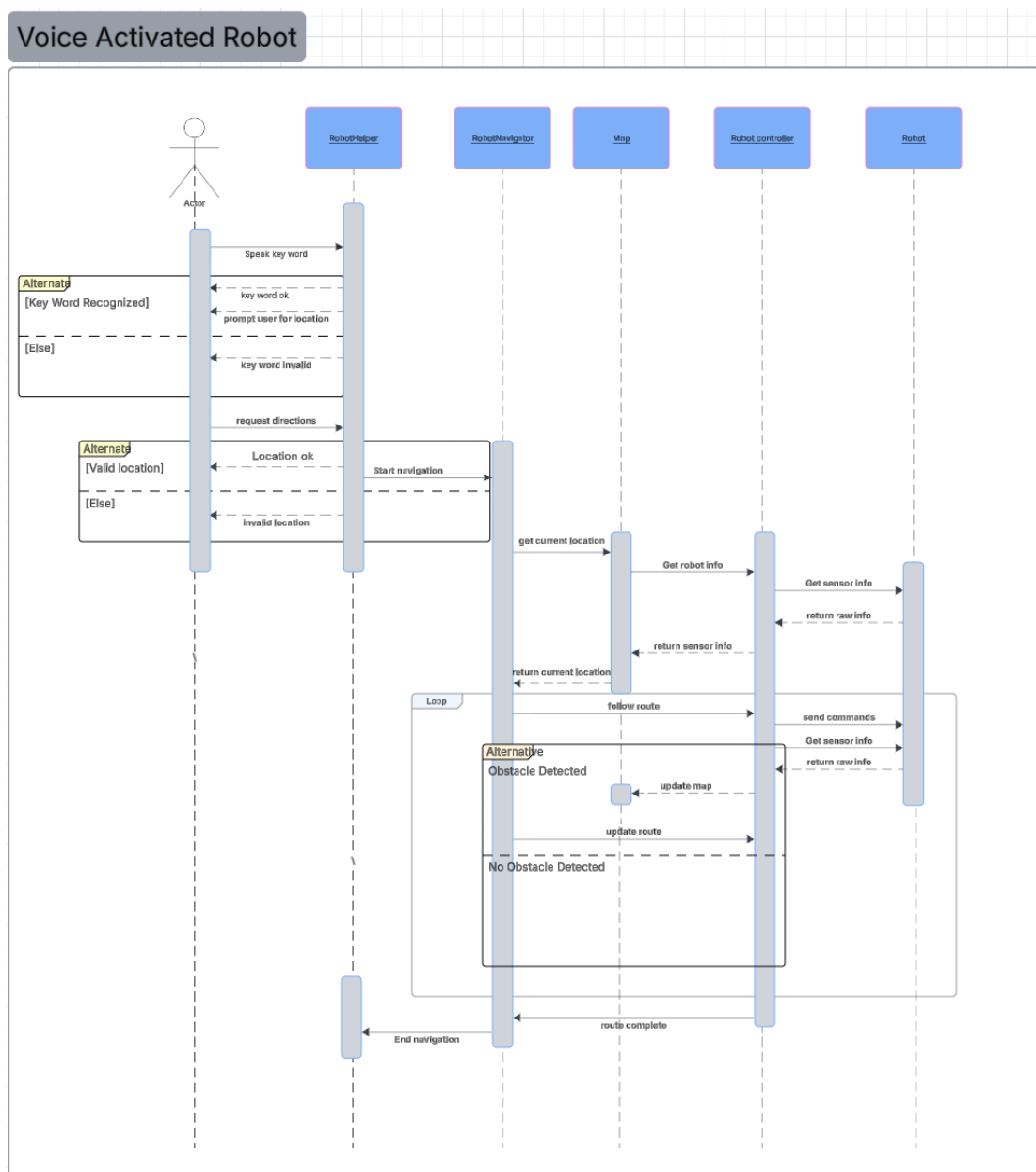


Figure 10: Sequence Diagram

The sequence diagram shown above illustrates the order of interactions from the mobile app to the robot. To start, the user will open the mobile application on their iOS device and speak the keyword. Once verified, the application will prompt the user for a command. Once the command is confirmed to be a valid command, the application will send the necessary information to the classes running on the Rubik Pi and will begin the navigation process. If the key word or command is not recognized, the app will return to its initial screen. The RobotNavigator classes

will request the robot's location from the Map class. The Map class will request the expected format of data from the RobotController. The RobotController will grab the raw data collected from the sensors attached to the Rubik Pi and convert the data into the required formats expected by the Map class. After localizing the Map class will return the robot's current location. Then the system enters a loop where RobotNavigator generates a route and sends a request to the RobotController to follow the route. While the robot is following the route it will send the data collected to the RobotController. The RobotController will detect any obstacles based on the data collected. When obstacles are detected the RobotController will send a request to the Map class to update and the RobotNavigator will use the updated map to update the route. This loop will complete when the route is complete. Once the destination is reached RobotNavigator will signal to the RobotHelper that navigation has ended.

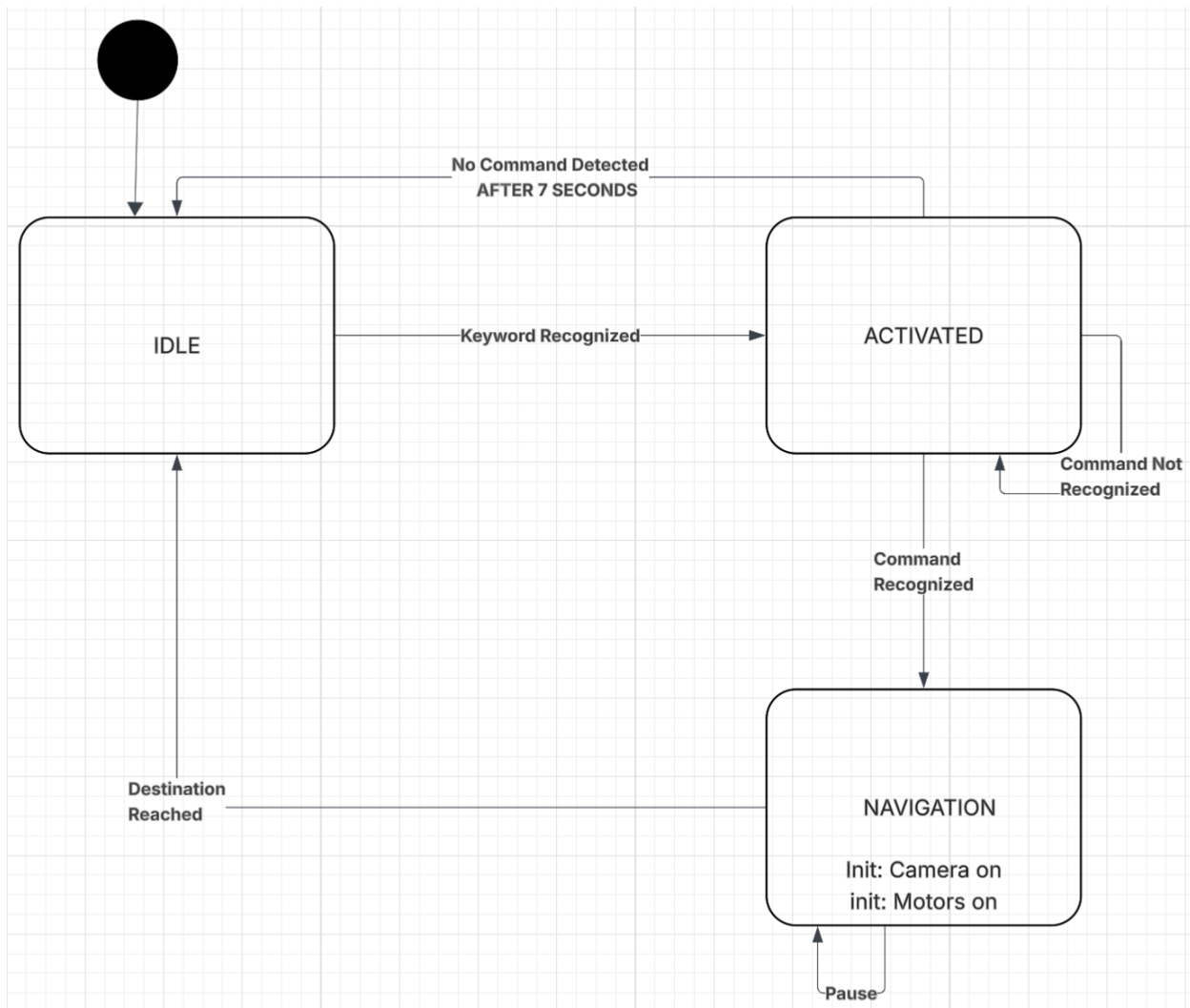


Figure 11: State Diagram

The state diagram above shows the three main states of our system. Initially our system is in idle mode and listens for a pre-defined keyword (e.g “Robot”). Once the keyword is recognized, the system will transition to Activated mode. Activated mode will then listen for a command. If the keyword is not recognized or if no speech is detected after seven seconds of transitioning to activated mode, the system will return to idle mode. If a command is not recognized, the system will remain in activated mode until a command is recognized. Once a command is recognized, the system will transition to navigation mode. The navigation state will activate the camera and motors. Navigation will also pause if the user is out of proximity. The system will return to idle mode once the destination is reached.

6.5 Design Alternatives and Design Rationale

One design alternative considered for the architecture model used for our system was the model view controller (MVC) model. Currently we have decided to model our system using layered architecture. We decided to use layered architecture because of the clean separation of responsibilities between layers. This separation of responsibilities allows for isolated changes and modifications to each layer, to improve the maintainability and testing of our code. Additionally, although we have pivoted in our approach to the project to incorporate a mobile application, the application does not directly control or communicate with the layer responsible for controlling the robot's movements. If our system had allowed the mobile application to directly control the robot, it may have been more accurate to represent our system using the MVC model. The MVC model is beneficial for representing frequent UI changes shown to the user; however, for our project we require very minimal UI changes. The mobile app's main responsibility is to use voice recognition to verify keywords, including destinations. Once navigation begins, the user will not need to interact with the app and will only need to follow the robot to the intended destination. Due to these reasons, we ultimately decided our system would be best represented using the layered architecture model.

We considered many different structural patterns such as the Snapshot pattern for saving user commands in the event they are to disconnect and reconnect to the robot in a certain timeframe. Due to the nature of a Bluetooth connection and the functionality of the robot, we have decided against this pattern and decided to go with the state pattern instead. This ensures that if the user disconnects from the robot, it will return to the keyword state, requiring the user to say the keyword again. The state pattern seems like a much better fit as our application has three separate states. The app starts in the keyword state where it waits for the user to say the keyword. The app then enters the command state where it waits for the user to give a command, such as “I would like to go to...”. This state then transfers the instructions from the operation to the robot.

After the command state, the app enters the navigation state where the Bluetooth connection is continuously checked and if the user is getting further away, the robot will stop, and the user will be prompted by the robot through TTS to return to range of the robot. Resetting to the keyword state prevents new users from being confused as returning users.

7. System Implementation

7.1 Programming Languages & Tools

In our project, we will primarily be using Python as our programming language due to its easy compatibility with our models and their porting through our Rubik Pi 3. It will mostly be used to call libraries we have downloaded like Nav2 and SLAM Toolbox and create wrappers for hardware to be used with ROS. We will also be using Python to implement the other models that we will be using on our Rubik Pi. Additional tools that will help us during this project will be the libraries included in Swift that will allow us to use voice recognition and Bluetooth connections. The whole development process will be falling under Ubuntu Linux, which is the operating system used by the Rubik Pi. Visual Studio Code will also be used mostly for testing and debugging. All these tools will help us, but our main hurdles will be trying to integrate every model with each other to make sure all of them will be running smoothly and at the same time when the robot is activated and running.

7.2 Coding Conventions

In the development process, keeping up the standard for proper coding conventions is a big priority. The code that we will be producing will be readable as well as consistent. This is important when we branch off to modular programming. It will be efficient and ideal to use modular programming as we will have a lot of distinct functions that each need a lot of effort and care. Knowing this, it is important that we keep our coding guidelines consistent. This can include using specific words when naming functions, so they can be easily understood when a new programmer looks at it. We will also be heavily using comments throughout our code so that even if a large function looks overwhelming, it can be easily deducted and understood from the comments made by the original writer; this makes it easy to find bugs or rewrite code if needed. Of course, with the code itself, it's important to keep formatting and syntax in mind; clean code equals clean reading. Each line of code or each function should be easily traceable as it goes deeper into the logistics of each model.

7.3 Code Version Control

To keep up with the updated code as the project progresses, we will be using Github as our main repository source, providing tools for us to update project history or to track different issues throughout the code. We want to make sure that our code is organized for an easy location and somewhere that has all our code in the unfortunate potential event that it gets lost somewhere else. With GitHub, we will also be able to branch out our files to test new features without having to change the original source code. Overall, it is a clean and reliable way to ensure code version control.

7.4 Implementation Alternatives and Design Rationale

When developing this application, we considered porting different AI models from Qualcomm AI hub to our development board such as different speech recognition models, object detection models, and facial recognition models. We planned for the system to recognize the user using a facial recognition model and track the user's face to determine their distance from the robot during navigation. An issue we encountered with this approach was that our WaveRover robot was too low to the ground to be able to accurately detect the users' face from the angle of the camera. Our solution to this issue, due to the limitations of our robot's height, is to instead use a mobile application. The mobile application will allow the user to connect to the Rubik Pi via Bluetooth, and our system will use the strength of the Bluetooth signal to determine the users' distance from the robot during navigation. Additionally, we had issues porting a speech recognition model to the board, and the mobile application is capable of using Swift libraries for speech recognition. By incorporating the mobile application, we will be able to bypass the need to port a speech recognition model to our Rubik Pi. Lastly, we initially planned on using object detection models to recognize potential obstacles; however, we will instead use our RPLidar sensor as our main means of determining whether the path in front of the robot is open or not. As a stretch goal we plan on implementing an object detection for objects that are detected under the lidar sensors field of view since the sensor can only view obstacles on a 2D plane on top of the robot. Ultimately, we have decided to incorporate a mobile application to free space and power on our board and to create a more user-friendly experience when using our navigation tool.

7.5 Consistency of Code Implementation on Chosen Design Patterns

Throughout the project, we maintained a consistent application of our chosen architectural patterns across the mobile application. Once MVVM and the State pattern were established as the core structure, they were applied together rather than being used in isolated parts of the code. In the mobile application, all major features were implemented with the MVVM structure, ensuring UI logic remained within the View, application logic was in the ViewModel, and data structures were in the Model layer. This consistency prevented logic from being duplicated or scattered across the app and made room for expansion. The State Pattern was also applied consistently whenever behavior needed to change based on system conditions, both in the apps control flow and in robot behavior. New functionality was integrated by simply extending the existing state structure to ensure the system behavior would remain the same regardless of feature addons. By consistently applying these patterns we avoided inconsistent implementation styles across different parts of the project. This made our code easier to navigate and ensured our app and robot followed the same logical structure when handling state changes and user commands. Overall, the consistency in applying these patterns contributed to a more maintainable and scalable system where features could be integrated without disrupting existing functionality. For the adapter pattern, the client is a model we are using, Nav2, which is not

visible in our architecture since we did not write the code for it. Essentially the bring-up is the interface which launches cmd_serial to allow Nav2 to write commands to the WaveRover. This works through subscribing to Nav2 and publishing the correct commands required when Nav2 publishes the movement commands.

7.6 Analysis of Key Algorithms

A core part of the system is the processing of voice input from the mobile application into robot commands. Since speech recognition can produce continuous input, the system requires a controlled algorithm to interpret commands and ensure they are only acted upon when the system is in a valid operational state. To handle this, we designed a voice command processing algorithm that works closely with the system's state architecture. Instead of immediately executing all recognized speech, all input is first evaluated based on the current application state (idle, listening, command received, or navigation). This prevents unintended actions and ensures predictable robot behavior. The algorithm also acts as a bridge between UI and robot communication. Once a valid command is detected, it is passed to the robot only if the system is in the correct state, ensuring safe and controlled execution of commands.

Pseudocode:

ProcessVoiceCommand(inputText):

switch currentState:

CASE Idle:

If inputText contains "robot"

TransitionTo(Listening_State)

CASE Listening_State:

If command is valid:

TransitionTo(Command_Received_State)

sendCommandToRobot(command)

Else:

ignore input

CASE Command_Received_State:

Ignore input

CASE Navigation_State:

If inputText contains "stop"

```
SendCommandToRobot("Stop")
```

```
Else
```

```
ignore input
```

The control_loop algorithm for our robot controller runs through the code many times per second. This allows our robot to be constantly able to move while still checking things like speed and safety parameters. This algorithm bridges velocity commands from the Rubik Pi to the Waverover. Instead of sending speed changes directly to the robot, it gradually moves the speed toward the requested one using smoothing. It allows the rover to drive in a way that looks smooth and realistic for a high-level robot. It also allows the robot to stop when a command has not been sent in a while, preventing the robot from moving when not desired.

Pseudocode:

1. Get the current time.
2. Calculate how long it has been since the last velocity command was received
3. If the last command is too old:
 - Set target forward speed to 0
 - Set target turning speed to 0
4. Smooth the current forward speed toward the target forward speed.
5. Smooth the current turning speed toward the target turning speed.
6. If the current forward speed is close to 0:
 - Set current forward speed to 0
7. If the current turning speed is very close to 0:
 - Set current turning speed to 0
8. If the last command is too old:
 - If the robot is not already stopped AND both current speeds are now 0:

```
Send a stop command to the WaveRover
```

```
Mark the robot as stopped
```

```
End Loop
```

9. Else:
 - Convert the current forward and turning speeds into left/right wheel commands
10. Send the drive command to the WaveRover over USB serial
11. Mark the robot as not stopped

8. System Testing

8.1 Test Automation Framework

8.1.1 Steps for Installing Test Framework

At our current state of our project, automated testing tools are not used; all testing is performed manually. The development environment was set up all through Visual Studio Code using Python 3 on the test machine and used the “requests” library to send JSON commands over Wi-Fi to the robot. Python scripts were written to directly control the robot by sending HTTP requests containing JSON commands to the robot’s IP address. The developmental machine was connected to the same Wi-Fi network as the robot to enable communication and command execution.

Another method of testing we are using is also not using automated testing tools but viewing changes on the map in RViz2 to verify that hardware is correctly configured with our ROS environment. To create this test framework, ROS must be installed on the development board along with RViz2 to view how the system detects changes to the environment and how it views changes in the robot’s pose. RViz2 has functions like measure which can allow us to see how far away our lidar sensor thinks an object is relative to the sensor's position. To use hardware with ROS, a wrapper must also be created for each piece of hardware which can usually be found online. In addition to ROS and RViz2, we also used Teleop Twist Keyboard to control the robot manually, which helps with testing things like whether the IMU is correctly configured with the system.

8.1.2 Steps for Running Test Cases

Test cases were executed manually by running the Python scripts to send commands to the robot and observing its physical responses. Movements such as moving forward or turning were triggered using the corresponding Python functions. In addition, the mobile application was tested manually by voicing the wake word “robot”. The app was verified to correctly detect the wake word and transition from the “idle” state to the “listening for commands” state. All results were recorded in real time to correct robot behavior and app functionality. Another way test cases were analyzed is by viewing the map in RViz2 on the device to see that the system is detecting movements created by the robot. Also, through testing hardware we receive to see how accurate it is, like the lidar sensor being tested using the measure function in RViz2.

8.2 Test Case Designs

TC-001: This test case verifies that the robot correctly interprets and executes a JSON command to move forward.

(Refer to Table 8.2.7 in Appendix T)

TC-002: This test case verifies that the robot correctly interprets and executes a JSON command to turn left.

(Refer to Table 8.2.8 in Appendix T)

TC-003: This test case verifies that the robot correctly interprets and executes a JSON command to turn right.

(Refer to Table 8.2.9 in Appendix T)

TC-004: This test case verifies that the robot correctly interprets and executes a JSON command to move backward.

(Refer to Table 8.2.10 in Appendix T)

TC-005: This test case verifies that the application correctly detects the keyword 'robot' and transitions from the idle state to the 'listening for commands' state.

(Refer to Table 8.2.11 in Appendix T)

TC-006: This test case checks the accuracy of the lidar sensor to ensure that it will be able to detect objects before coming into contact with them.

(Refer to Table 8.2.12 in Appendix T)

TC-007: This test case checks that the IMU can accurately detect a change in direction when the robot is turning right

(Refer to Table 8.2.13 in Appendix T)

TC-008: This test case checks that the IMU can accurately detect a change in direction when the robot is turning left

(Refer to Table 8.2.14 in Appendix T)

TC-009: This test case checks that obstacle avoidance is working correctly and handles avoidance efficiently.

(Refer to Table 8.2.15 in Appendix T)

TC-010: This test case checks that the system is creating realistic paths to make its way to the destination.

(Refer to Table 8.2.16 in Appendix T)

8.3 Test Execution Reports

TC-001: Executed a python script in VS Code to send a JSON "move forward" command to the robot over Wi-Fi and observed the robot successfully move forward. Failed once due to batteries being dead.

(Refer to Table 8.3.1 in Appendix TE)

TC-002: Executed a python script in VS Code to send a JSON "turn left" command to the robot over Wi-Fi and observed the robot successfully turn left. Failed once due to batteries being dead.

(Refer to Table 8.3.2 in Appendix TE)

TC-003: Executed a python script in VS Code to send a JSON “turn right” command to the robot over Wi-Fi and observed the robot successfully turn right. Failed once due to batteries being dead.

(Refer to Table 8.3.3 in Appendix TE)

TC-004: Executed a python script in VS Code to send a JSON “move backward” command to the robot over Wi-Fi and observed the robot successfully move backward. Failed once due to batteries being dead.

(Refer to Table 8.3.4 in Appendix TE)

TC-005: Spoke the wake word “robot” while the app was in idle state and confirmed that I transitioned to the Listening for Commands state. Failed once due to not speaking clearly.

(Refer to Table 8.3.5 in Appendix TE)

TC-006: Placed a marker 5 centimeters from the robot and ran the measure function in the RViz2 map on the marker. The marker was detected 5.34 centimeters away from the marker in RViz2 passing the test. Failed once due to lidar not starting correctly. Still unsure about the issue, but it works sometimes when the start-up file is run and does not work other times. This issue is new due to adding lidar odometry.

(Refer to Table 8.3.6 in Appendix TE)

TC-007: Ran Teleop Twist Keyboard while RViz was running along with the IMU wrapper for ROS. After turning the robot right using the keyboard command, a change was viewed on the map where the robot had turned right passing the test. Failed once due to batteries being dead in the robot.

(Refer to Table 8.3.7 in Appendix TE)

TC-008: Ran Teleop Twist Keyboard while RViz was running along with the IMU wrapper for ROS. After turning the robot left using the keyboard command, a change was viewed on the map where the robot had turned left passing the test. Failed once due to batteries being dead in the robot.

(Refer to Table 8.3.8 in Appendix TE)

TC-009: Ran RViz and Nav2 then set up a box that was not present in the pregenerated map. Placed a goal behind the box to ensure the robot had to avoid the obstacle. Observed if the robot avoided the obstacle in a reasonable way. Failed once due to object detection parameters not being inflated enough.

(Refer to Table 8.3.9 in Appendix TE)

TC-009: Ran RViz and Nav2 and then placing a goal in front of the robot. The purpose of this is to observe the robot making its way to a goal and drawing efficient pathing. Failed once due to a bug causing the robot unable to turn right.

(Refer to Table 8.3.10 in Appendix TE)

8.4 Meeting Functional Requirements

The functional requirement UF-A has been refined into system requirements SF-A-01 and SF-A-02. SF-A-01 has been completed, and the system can successfully create viable paths from its starting point to the goal. For system requirement SF-A-02, text to speech has been implemented. (Refer to Figure 4.6 in Appendix R)

The user functional requirement UF-B has been refined into three system requirements. SF-B-01, SF-B-02, and SF-B-03. These system requirements have been completed. The mobile application recognizes Bluetooth connection and parses the user's spoken commands.

(Refer to Figure 4.7 in Appendix R)

The user functional requirement UF-D has been refined into system requirements SF-D-01 and SF-D-02. These system requirements have been completed. The system uses a SLAM model to create a map with obstacles and automatically adjusts the path to avoid collisions, ensuring the user can follow the robot safely.

(Refer to Figure 4.8 in Appendix R)

The user function requirement UF-F has been refined into system requirements SF-F-01 and SF-F-02. Both these system requirements have been completed. The mobile application successfully listens for a keyword and transitions to a different state when the word is recognized.

(Refer to Figure 4.9 in Appendix R)

8.5 Meeting Non-Functional Requirements

The non-functional requirement UP-03 have been refined into the system requirements SP-03-01, SP-03-02, SP-03-03, and SP-03-04. The system responds to Bluetooth connection quickly and pauses when Bluetooth signal is lost. The system responds to issued commands and responds quickly after the command is received.

(Refer to Figure 4.10 in Appendix R)

The non-function requirement UE-01 was refined into system requirement SE-01-01. This system requirement has not yet been implemented.

(Refer to Figure 4.11 in Appendix R)

9. Challenges & Open Issues

9.1. Challenges Faced in Requirements Engineering

The challenges faced involve both scheduling and setting up the development environment. For the issue of scheduling, the entire group has classes fulltime and majority work on top of that. This makes it difficult to plan a meeting time to meet with our industry mentor. Luckily, our mentor was able to meet in the middle of all our busy schedules, and we are now able to communicate through Teams once a week. For the issue of setting up the development environment, none of us in the group have ever worked with development boards before in the past. This makes setting them up a large challenge for us. This challenge is made worse by the development board; it being a very new product that was released this year with minimal documentation. Due to this we are still working to get the environment set up with only one member of the group being able to connect only through ADB shell. Another challenge is coming up with a plan for this open-ended project with instructions to be interpreted by the group instead of the mentor. This level of creative freedom is great but has also added to the challenge as it can be difficult to decide how large we would like the scope of this project to be.

9.2 Challenges Faced in System Development

One of the first major challenges we faced as a group during this project was the initialization and the onboarding process of getting the Rubik Pi 3 to work on our personal computers. The development board had limited instructions and documentation to reference us to use, so we had trouble deciding which operating system to implement for our specific project. With the help of experts on our specific development board, we were able to flash our operating system Ubuntu on our boards.

Learning how to evolve our development board using models is something that we had to learn as well. The idea of using software models for our project wasn't entirely unknown, but it was something that we had to familiarize ourselves with. We weren't used to testing the success of our project through commands and not through conventional coding, compiling, and running. Through our models we also had to learn how to thread so we can have multiple models running at the same time through the support of our camera hardware, for example having a motion tracking model on at the same time as our object detection model.

Another aspect of our project we had to face was the learning curve of creating a fully functional iOS application for our voice detection system for the robot. We learned how to use swift and its given libraries to successfully implement voice detection and keyword detection to have the robot receive commands based on specific keywords. The mobile app will implement the requirement of having the robot stop after a certain distance is reached between the user and the robot using Bluetooth connection.

After installing Orb Slam 3 it was clear it was not running efficiently and was not compatible with the existing Qualcomm drivers which utilize the NPU. There were also the added compatibility issues with our RGB module 3 Raspberry Pi camera with our development board. Our camera was provided by Qualcomm but lacked driver support. Additionally, the camera lacked depth perception, which is a crucial piece of functionality that provides required data to create a map using a slam model and to use navigation software. Because of this we decided to switch to SLAM Toolbox which is designed to work with the boards' existing drivers and will run much more efficiently on the system. We also decided to use a LiDAR sensor which is commonly found on most autonomously moving robots, utilizing lasers and laser odometry to identify foreign objects that stand in front of our robot.

While integrating Nav2 and Slam Toolbox we encountered issues with hardware limitations. Our Lidar sensor consistently failed to start causing our software to crash. Although we are unable to resolve this hardware issue, we found a way around this problem. In order to get the Lidar sensor to start scanning before launching our system, we let the Lidar run for 30 seconds on a stable surface, once stable we launch our system. Additionally, our robot was unable to complete turns request by Nav2. We found out our robot is unable to perform turns at low speeds. To resolve this issue, we increased the speed for turns and kept the speed lower for forward and backward commands. Another issue we encountered was odometry was inconsistent with the location of our robot in the map. Alone, our Lidar sensor is an unreliable source for odometry however fused with the Imu it provides better odometry.

9.3 Open Issues and Ideas for Solutions

Currently, we are very close to finishing our whole system. We have all the necessary practices and hardware required to finish our system. Our current battle is trying to connect the iOS application to our navigation stack. We just need to work on configuring our code, so the navigation system knows what it means when the app sends a command through a user's input.

Our recent issue was creating a map on a larger scale. We have successfully tested mapping out small rooms with our system, and it works really well, creating a readable and cohesive map really fast. When trying it out on a larger room with more complex obstacles, the system seems to lag or glitch and sometimes either detects ghost obstacles, or the visualization fails to recognize where the robot is amongst the maps. We have found a solution by testing, and we have realized that the more we stop and turn, the more the robot gets confused by overexerting power, trying to figure out why it stopped moving or where it is. We have found that moving the robot in straight lines and mapping the perimeter first yields better map results. Taking it slow also helps, letting the visualization have more time to update its map.

10. System Manuals

10.1. Instructions for System Development

10.1.1. How to set up development environment

To set up the development environment it is necessary to have access to the Rubik Pi 3, currently no emulators for it exist or are widely available. You can purchase the Rubik Pi 3 online.

1. Once you have your Rubik Pi 3, access online documentation for the initializing and setup process for the Rubik Pi 3. Resources necessary for this project include the Rubik Pi 3 official website [3], Thundercomm documentation [4], and Edge Impulse documentation [6]

2. Plug in the Rubik Pi 3 using a USB-C cord to a power source of at least 5V.

- 2.1 Optionally, you can also plug in a keyboard and mouse through USB as well as a monitor through HDMI to work directly on the Rubik Pi 3 system.

3. Download the latest Ubuntu operating system image compatible with the Rubik Pi 3. For our project specifically, we used Ubuntu 24.04 as it was the most recent version.

- 3.1 Optionally, you can also download Ubuntu Desktop, allows you to access the internet and connect to Bluetooth and Wi-Fi easily.

4. Follow the documentation section on Qualcomm to flash Ubuntu 24.04 on the Rubik Pi 3 [3]. The section is titled “Flash Canonical Ubuntu 24.04 using Qualcomm Launcher”

5. After the flashing process is done, fully power the Rubik Pi 3 and connect to the same network as your initial machine.

6. To determine if Ubuntu is running correctly, either use the keyboard/mouse/monitor to log into the board to verify connection, or you can SSH using a local terminal or VSCode that supports SSH.

7. Now that you are fully connected, you are now free to install or import any project dependency and begin your development process

10.1.2. Notes on System Further Development

Our system uses the SLAMTEC RP Lidar A1 and its open-source driver [19]. However, the system could be developed to work with other 2D Lidars or 3D Lidars for performance quality. The Lidar should be connected through i2C, and the correct port should be defined in the node launch file as well as the odometry, localization, and navigation bring up. Additionally, drivers for the hardware components are open source and could be replaced by other open source or custom drivers as long as the topics published by the Ros nodes match the expected topics of the overall system. These topics can be found in the Nav2 yaml in the rover bring up config folder. The Nav2 yaml provides settings for speed of the robot, error allowance, and navigation behavior. These settings can be adjusted to change the robot's behavior during navigation. One limitation of the current system is that obstacles below or above the 2D Lidar may go undetected. The system could be further improved by the integration of a 3D Lidar as mentioned above or a camera with depth perception. The system could fuse Lidar and camera perception values to improve obstacle avoidance and accuracy. This would require that the sensors be aligned and

publishing at the same rate. The fused topic should replace the original scan topic in the above-mentioned files.

10.2. Instructions for System Deployment

10.2.1. Platform Requirements

To successfully run our system, there are a couple of prerequisites required. First, of course, you need all of the necessary hardware.

- Rubik Pi 3 Development Board
- A1 – RP LiDAR Sensor
- Wave Rover 4WD Robot
- Portable Battery Bank >5V
- SparkFun Qwiic pHAT v2.0 Adapter
- Adafruit 9-DOF Absolute Orientation IMU Fusion Breakout
- USB/Bluetooth Keyboard & Mouse
- HDMI Connected Monitor
- Portable Speaker with 1 x 3.5mm Audio Jack
- Mounted 3 Layer Hardware (3D Printed or Self Made)
- Laptop/Computer with XCode Installed
- IOS Device

To connect the hardware system together, below are the steps to connect all the robot related hardware for the working system.

1. Attach the mounted hardware on top of the Wave Rover 4WD Robot
- 1.1 Attach the IMU in front of the robot through the preferred screw hole.
2. Place the battery bank on the lower shelf closest to the robot, and plug it in through the USB-C port on the Rubik Pi
3. Attach the Qwiic Adapter on the 40-pin plug on the Rubik Pi and plug in the portable speaker through the designated plug. Then place Rubik Pi in the middle layer of the mounted hardware.
4. Place the RP LiDAR sensor on the top layer of the mounted hardware, ensuring its stable
5. Plug in the RP-LiDAR to the Micro-USB port on the Rubik Pi
6. Plug in the robot cord into the USB port on the Rubik Pi
7. Using the Qwiic adapter, plug in the cord to the screwed in IMU

Now that your hardware is all connected and accounted for, make sure you have all the necessary software components listed below. Specific installations instructions will be provided later.

- Ubuntu 24.04
- Ubuntu Desktop

- ROS2 Jazzy
- Nav2 / SLAM ToolBox
- Piper TTS
- IMU Driver
- Lidar Driver
- XCode

10.2.2. System Installation

First, we are assuming that you have successfully flashed Ubuntu on your Rubik Pi. To install Ubuntu Desktop, you will need to download it following the instructions found on the official Rubik Pi documentation Ubuntu setup page. When you are finished installing, make sure you update the system.

```
sudo apt update && sudo apt upgrade -y
```

Next, you will need to install ROS2 Jazzy. To do so, set up the following commands and set your environment, then install your build tools.

```
sudo apt install software-properties-common
sudo add-apt-repository universe
sudo apt update
sudo apt install ros-jazzy-desktop

echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc

sudo apt install python3-colcon-common-extensions
```

Next, you will need to configure your ROS 2 workspace, so you can use all of your packages provided by ROS. Afterwards you will source the workspace which ensures future usage.

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
colcon build

echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Finally, to install Piper TTS, you will need to simply install python pip as well as clone the open source github. It is worth mentioning that configuring your speaker or your output device will depend on what you are using.

```
sudo apt install git python3 python3-pip
git clone https://github.com/rhasspy/piper.git
cd piper
```

To install Nav2 follow the official documentation for ros2 jazzy found on the ros page [20].

To install Slam Toolbox follow the official documentation for ros2 jazzy found on the ros page[21].

To install the driver for the imu [17]:

```
cd ~/ros2_ws/src
git clone https://github.com/flynneva/bno055.git

cd ~/ros2_ws
colcon build
```

To install the driver for the Lidar sensor [19]:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src

cd ~/ros2_ws/
source /opt/ros/<rostdistro>/setup.bash
colcon build --symlink-install

source ./install/setup.bash
$echo "source <your_own_ros2_ws>/install/setup.bash" >> ~/.bashrc
$source ~/.bashrc

sudo chmod 777 /dev/ttyUSB0
```

Before each use, you will need to run the command to source and open the environments said above, both jazzy and your ros workspace. To check for Rviz, simply run the command:

Rviz2

On your laptop or external device, make sure you download XCode by following the instructions on online documentation. Afterwards make sure you connect your iOS device for deployment and make sure it can connect to an advertising Bluetooth Signal.

10.3. Instructions for System End Users

Now that everything is set up correctly, the actual usability for our system was designed to be really easy. The only thing the user needs before using our system is the app itself. Once the user has the app, they need to open it and will be prompted to pair with the robot via Bluetooth. Our app will indicate and tell you if that pairing is successful.

Once paired, the app will ask the user if they are ready to start the voice activation process. Once the user agrees and starts the prompt, the user will need to say a keyword for the mobile application to transition to a listening state. The activation word is 'Robot'. Once the keyword is activated, the robot will start listening for a command. The user will then vocally ask for directions to a specific place, which are predetermined and provided through the app. Once the robot recognizes a valid destination, the robot will start routing and moving to that location, avoiding obstacles on the way. The user will follow the robot as it moves along its path. There is an RSSI indicator on the app which tells the user the connection strength, prompting the user to get closer when it gets too low. There is a case where if it reaches a threshold which indicates the user is too far from the robot, the robot will stop. Once the user and the robot reach the destination, the app will prompt the user if it would like another destination. If the user says yes, repeat the voice activation process. If not, the app will stop listening and the user will have finished the process.

11. Conclusion

11.1. Achievement

This project's primary objective of creating a cohesive map of an environment and having a voice activated guide robot to navigate that environment through voice command in an iOS application has been achieved. We were successfully able to implement ROS2 on our Rubik Pi, allowing us to use its inherent packages to help with robot communication, map visualization, and navigation. Using other hardware data, we were able to generate a 2D map of the environment and its surrounding area.

We were also able to create an iOS application that connects to the Rubik Pi via Bluetooth. Our iOS application can take in user input through voice commands and parse through the sentences to detect keywords that initiates certain movement instructions in the Rubik Pi to move the robot.

Our system can collect LiDAR and IMU data and process it in real time to support the navigation system through obstacle detection, localization, and map monitoring. Through the combination of these different systems, we successfully achieved autonomous navigation to guide new users without external help.

11.2. Lessons Learned

Montserrat:

In this project, I learned how to flash an operating system onto a development board. Additionally, I learned how to connect different hardware components to our development board and how to troubleshoot hardware issues such as a Lidar sensor not starting. This project allowed us to come up with creative solutions to resolve hardware issues, software issues, and to ultimately achieve our end goal. I learned about different SLAM models and navigations stacks and how to use open-source packages on our board. I spent a lot of time working in the Linux terminal and understanding how to source and use a ROS2 workspace. I gained experience working with different ROS packages such as Slam Toolbox and Nav2 as well as ROS2 features such as creating ROS packages and using ROS nodes. Beyond software and hardware, I gained more practice in software management practices and system documentation.

Beau:

Through working on the Rubik Pi, I learned about how to work with development boards since this was my first time working with one. We were given a unique experience to test out a board which is relatively new and had very little documentation and forums speaking about it online. I also learned how to work with an Ubuntu operating system as well as resolving and diagnosing issues within it. This project has helped me with diagnosing issues which would not be detected through code but through observation like bugs causing the robot to only turn left. I also learned how to work with ROS, which seems to be a very important framework to know in the robotics world. I am glad I have learned it since it seems like a very good framework for projects which

can handle parallel processing automatically. I also gained a lot more experience coding with Python, which I had very little experience previously. I also learned that when working with hardware coding is a small fraction of the work when compared to testing all the different pieces individually and then testing again when they are working together. Through our industry mentor I picked up many different skills like not letting time waiting for hardware get in the way of working on what you can. He also taught me about how wires can be chained together so that they can both connect to the same Arduino pins which I would never have thought could be done before this project.

Jacob:

Throughout this project I have learned many skills such as IOS development, working with development boards, the contrast between hardware and software, and working as a team. Prior to this project, I had experience with android development, but this was my first time using Xcode to develop an app for IOS. I personally preferred using Xcode over android studios due to easy usability and the wide range of libraries iOS provides which are very easy to import. This was also my first time working with hardware, and it introduced our team to many new challenges when troubleshooting our system. We realized that a lot of hardware has limitations and taught us to work around these problems because they were out of our control. Also, when testing our system, we constantly had to troubleshoot outside of our code such as altering our environment, positioning of hardware, or considering using different hardware to satisfy our requirements and features. While sometimes overwhelming, keeping a balance between the software and hardware was a challenging task and required a lot of planning and testing. Specifically, using the developmental board posed a big challenge to our team because it was the first time any of us had worked with one and required a bit of setup before we could write any code. I believe our experience with our developmental board has improved our understanding of how these systems work, which are quite common in a lot of projects in the real world. Lastly, I have learned a lot about the importance of working as a team. Through many sprint reviews and daily meetings, I now have experience collaborating with a team on a long-term project.

Jonathan:

In this Capstone project, I was given practical experience with embedded systems and robotics with its interaction between software and hardware through a development board. This was my first project with a development board and my first introduction to the Rubik Pi as well as the Raspberry Pi. Through this introduction I learned how to flash an operating system to it and how to deploy different models onto it. I also gained a deeper understanding of how Bluetooth works and how two compatible devices can connect to each other and exchange data. While working on this project, I also had practical experience in learning how IMU and LiDAR data contribute to a navigation system and how a robot knows its environment and obstacle detection. This enhances how I see more technically advanced versions of this robot perchance Waymos or Roombas and also enhances my robotics knowledge overall.

11.3. Acknowledgment

Our Capstone project was sponsored and financially supported by Qualcomm, and we had an industry mentor we met weekly. Our mentor provided meaningful feedback each week based on our progress and provided helpful advice for approaching our project. Our mentor also provided insight into industry standards and how to present our product. In the beginning we struggled to flash an operating system to our board and Qualcomm set up a meeting to provide a tutorial on how to flash the board. Furthermore, when we ran into issues during winter break with our hardware, Qualcomm once again set up a meeting with their team to go over compatibility issues and to answer any questions about the board we might have had.

In addition to our industry mentor, we also had a faculty advisor Dr. Simon Fan, who we also met weekly who provided encouragement and advice that helped keep us motivated throughout the duration of our project. Our meetings with our faculty advisor also gave us opportunities to ask questions about our project and probable solutions to problems we encountered.

12. References

1. “Qualcomm AI Hub.” Qualcomm AI Hub, aihub.qualcomm.com/models.
2. “SFSpeechRecognizer.” *Apple Developer Documentation*, developer.apple.com/documentation/speech/sfspeechrecognizer. Accessed 27 Nov. 2025.
3. “Rubik Pi.” *RUBIK Pi 3, a Lightweight Development Board Based on Qualcomm Dragonwing™ QCS6490 Platform*, 11 Oct. 2025, rubikpi.ai/.
4. “Get Started.” *RUBIK Pi Documentation*, www.thundercomm.com/rubik-pi-3/en/docs/rubik-pi-3-user-manual/1.1.0/quick-start/.
5. *Qualcomm Documentation*, docs.qualcomm.com/bundle/publicresource/topics/80-70018-50/overview.html.
6. “Thundercomm Rubik Pi 3 - Edge Impulse Documentation.” *Edgeimpulse.com*, Edge Impulse Documentation, 2025, docs.edgeimpulse.com/hardware/boards/thundercomm-rubikpi3.
7. Inc., Apple. “Swift.” *Apple Developer*, developer.apple.com/swift/. Accessed 27 Nov. 2025.
8. “Connecting to Bluetooth Peripherals in Swift Playgrounds.” *Apple Developer Documentation*, developer.apple.com/documentation/playgroundbluetooth/connecting_to_bluetooth_peripherals_in_swift_playgrounds. Accessed 27 Nov. 2025.
9. Campos, Carlos, et al. “Orb-Slam3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map Slam.” arXiv.Org, 23 Apr. 2021, arxiv.org/abs/2007.11898.
<https://arxiv.org/abs/2007.11898>
10. “WAVE ROVER Flexible and Expandable 4WD Mobile Robot Chassis, Full Metal Body, Multiple Hosts Support, with Onboard ESP32 Module | WAVE ROVER.” *Waveshare.com*, 2025, www.waveshare.com/wave-rover.htm.
11. Open Robotics. “ROS.org | Powering the World’s Robots.” *Ros.org*, 2020, www.ros.org/.
12. “Ubuntu on Qualcomm® IoT Platforms.” *Qualcomm.com*, 2025, www.qualcomm.com/developer/software/ubuntu-on-qualcomm-iot-platforms.

13. “Layout: Forward Target:
https://developers.google.com/mediapipe/solutions/vision/face_detector/ Title: Face Detection Parent: Mediapipe Legacy Solutions Nav_order: 1.”
14. “NAV2.” *Nav2*, docs.nav2.org/. Accessed 20 Jan. 2026.
15. “Introduction.” *Slam_toolbox - ROS Package Overview*, index.ros.org/p/slam_toolbox/. Accessed 20 Jan. 2026.
16. “Teleop_twist_keyboard — Teleop_twist_keyboard: Rolling 2.4.1 Documentation.” *Ros.org*, 2025, docs.ros.org/en/ros2_packages/rolling/api/teleop_twist_keyboard/. Accessed 1 Mar. 2026.
17. “Bno055 - ROS Package Overview.” *Ros.org*, 2024, index.ros.org/p/bno055/. Accessed 27 Apr. 2026.
18. Github.com, 2026, github.com/MAPIRlab/rf2o_laser_odometry/tree/ros2/src. Accessed 27 Apr. 2026.
19. Slamtec. “Sllidar_ros2/README.md at Main · Slamtec/Sllidar_ros2.” GitHub, 2025, github.com/Slamtec/sllidar_ros2/blob/main/README.md. Accessed 27 Apr. 2026.
20. “Getting Started — Nav2 1.0.0 Documentation.” Docs.nav2.org, docs.nav2.org/getting_started/index.html.
21. “README — Slam_toolbox 2.6.10 Documentation.” Ros.org, 2021, docs.ros.org/en/humble/p/slam_toolbox/___README.html. Accessed 27 Apr. 2026.